RA26-Challenge 7: Enunciado Entrenamiento

ECS completo: gráficos, sistemas, sprites, oleadas, combate y fin de partida

STRONGMAN VS CHICKEN

Un tower defense por carriles. Los pollos entran por la derecha y avanzan hacia la casa por tres carriles. Con la cruceta eliges una casilla, con **A** abres el menú de compra y con **A** otra vez colocas un strongman, si tienes proteína suficiente. Pollo y strongman se pegan cuando se encuentran. Cada pollo derrotado da proteína. Si un pollo llega a la casa, pierdes. Si acaba la oleada y no queda ningún pollo, ganas.

El proyecto ya tiene la estructura ECS (gestor de entidades, sistemas y escena), el joypad, el cursor por estados, la proteína y la creación de pollos y strongmen. Vais a programar y a corregir las **10 piezas que faltan**. Juntas forman el juego: cuando terminéis, el juego funciona con vuestro código.

Este documento solo tiene los ejercicios
Lo que necesitáis para resolverlos está en los apuntes: cada ejercicio indica qué leer antes (S05.2 ECS, Challenges 5 y 6, «Cómo crear gráficos a mano»...). Leed eso y, después, resolved el ejercicio.

Trabajad como en clase

1. **En papel.** Pensad, dadle vueltas, intentad, fallad, probad de nuevo.
2. **En GBTelera.** Escribid el código, ejecutad, depurad y haced funcionar el juego.

Dos formatos de ejercicio

- **Implementar o completar:** la rutina está vacía (un stub provisional) o tiene parte del código. Escribid lo que falta donde pone `●●TU CÓDIGO●●`.
- **Corregir:** el código viene con errores de todo tipo (Ejercicio 6a). Encontradlos y arregladlos primero en papel y después en el proyecto.

Tiempo orientativo
Es el tiempo de resolver cada ejercicio entero: entenderlo, hacerlo en papel, pasarlo al ordenador y depurarlo. Repasar los apuntes que indica son unos 15-20 minutos más.

Ejercicios y reparto

Ej.	Pers.	Rutinas	Apuntes	Min.
1	1	`Tiles_*`, `gfx_init`	Gráficos a mano; Ch. 5 (gfx)	~50
2	1	`sys_render_one`	S05.2 (un sistema); Ch. 5 ej. 6	~45
3	2	`sys_cursor_init`	Ch. 5 (OAM); S01.3 (bucles)	~30
4	2	`sys_cursor_update`	Ch. 5 y 6 (sprites, coordenadas)	~45
5	3	`game_entidad_en`	S05.2; Ch. 5 ej. 1 (`alloc`)	~35
6	3	`game_comprar_strong` (corregir), `hud_update`	Ch. 5 ej. 1, 3; Ch. 6 ej. 4 (HUD)	~45
7	4	`game_oleada_init`, `game_oleada_update`	Ch. 6 ej. 5 (`enemies_spawn`)	~50
8	4	`sys_movement_one`, `sys_movement_update`	Ch. 5 ej. 3 (`sys_physics`)	~35
9	5	`sys_combat_one`, `sys_combat_update`	Ch. 5 ej. 10 (`collision_one`)	~50
10	5	`game_pollos_vivos`, `game_check_end`, `game_pantalla_final`	Ch. 5 ej. 11-12 (escenas)	~45
		Total		**~7 h**

Para un equipo de 5 personas (2 ejercicios cada una)
Unas 1,5 h de ejercicios por persona, más el repaso de sus apuntes. Cada persona trabaja en **sus propios ficheros**, así que al juntarlo casi no hay conflictos de merge:

Persona	Tema	Ficheros
1	Lo que se ve	`gfx/gfx.asm`, `sys/sys_render.asm` (solo `sys_render_one`)
2	El cursor y el menú	`sys/sys_cursor.asm`
3	Casillas y economía	`scn/scn_game_rules.asm` (ej. 5, 6a), `ui/hud.asm` (6b)
4	Los pollos	`scn/scn_game_oleada.asm`, `sys/sys_movement.asm`
5	Combate y final	`sys/sys_combat.asm`, `scn/scn_game_end.asm`

Orden: el Ejercicio 5 va primero
`game_entidad_en` (ej. 5) la usan los ejercicios 6, 7, 8 y 9. Hasta que esté hecha, su stub dice siempre «casilla libre» y el resto se puede escribir y compilar igual. La persona 3 debe empezar por ella y subirla cuanto antes.
Antes de nada, una persona añade el **Código común** (Paso 0, unos 20 minutos) y lo sube. El resto parte de ahí.

Organización del proyecto

Carpetas por función, como en los Challenges 5 y 6: `engine/` (servicios), `gfx/` (gráficos), `man/` (gestores), `sys/` (sistemas), `scn/` (escena y reglas) y `ui/` (HUD).

```
src/
├── main.asm                    Bucle principal (Paso 0)
├── engine/                     Servicios: pantalla, memoria, joypad, frames
│   ├── hardware.inc            (Paso 0)
│   └── frame.asm               (Paso 0, nuevo)
├── gfx/
│   ├── gfx.inc                 (Paso 0)
│   └── gfx.asm                 Ej. 1
├── man/man_entity.inc          (Paso 0)
├── sys/
│   ├── sys_render.asm          Ej. 2 (y código común del Paso 0)
│   ├── sys_cursor.asm          Ej. 3, 4 (nuevo)
│   ├── sys_movement.asm        Ej. 8
```

```
│   └── sys_combat.asm          Ej. 9 (nuevo)
├── scn/
│   ├── scn_game.asm            Escena: init y update (Paso 0)
│   ├── scn_game.inc            Constantes del juego (Paso 0)
│   ├── scn_game_rules.asm      Ej. 5, 6a (y código común del Paso 0)
│   ├── scn_game_oleada.asm     Ej. 7 (nuevo, con la oleada del Paso 0)
│   └── scn_game_end.asm        Ej. 10 (nuevo)
└── ui/hud.asm                  Ej. 6b
```

El juego en pantalla

La pantalla son 10 × 9 **casillas** de 2 × 2 tiles. Una entidad guarda su casilla en un byte, `E_POS = %ffffcccc` (fila y columna). Con `swap` se pasa de fila a nibble alto: `(fila << 4) | columna`.

	0	1	2	3	4	5	6	7	8	9
0	menú 0	menú 1	menú 2	menú 3	menú 4					
1										
2										
3										
4										
5	casa c0									entran
6	casa c1									entran
7	casa c2									entran
8	coste									prot.

- **Menú** (fila `FILA_MENU` = 0, columnas 0-4): un strongman por tipo. En modo menú el cursor va aquí.
- **Carriles** (filas de casilla `FILA_CARRIL0` = 5, 6 y 7). Fila del tablero `r` = fila de casilla `5 + r`. Los strongmen se colocan en las columnas 0-6 (en azul). Los pollos entran por la columna `COL_ENTRADA` = 9 y avanzan hacia la 0.
- **HUD** (fila de tiles 17): coste del strongman elegido a la izquierda y proteína a la derecha.

Cómo se juntan las piezas

`scene_game_update` decide el **orden** de los sistemas. Cada frame:

```
main:  input_update        Lee el joypad → wPad* (1 = recién pulsado)
       frame_tick          frame_counter++
       scene_game_update:
           sys_input_update      Cursor y menú (ya hecho). Comprar → game_comprar_strong (Ej. 6a)
           game_oleada_update    Salen pollos (Ej. 7)
           sys_movement_update   Los pollos avanzan, 1 casilla cada 64 frames (Ej. 8)
           sys_combat_update     Pollos y strongmen se pegan, cada 16 frames (Ej. 9)
           sys_render_update:    wait_vblank y, ya en VBLANK:
               sys_cursor_update     Los 4 sprites del cursor (Ej. 4)
               hud_update            Proteína y coste (Ej. 6b)
               sys_render_one        Cada entidad con RENDER (Ej. 2)
       game_check_end      ¿Ha acabado? (Ej. 10b) → Z: game_pantalla_final (Ej. 10c)
```

La vida de una entidad

- **Nace** con `game_chicken_spawn` (pollo: `RENDER | MOVE`) o `game_strong_colocar` (strongman: `RENDER`).
- **Se mueve** (solo los pollos): `sys_movement` cambia `E_POS` y marca `ESTADO_MOVIDO`. `sys_render` borra la casilla de la que viene (la de su derecha) y quita la marca.
- **Muere** con `game_matar`: `VIDA = 0` y solo le queda `RENDER`, así que ya no se mueve ni pega. En ese mismo frame, `sys_render` borra su casilla y la libera con `man_entity_free`.
- `game_entidad_en` y `game_pollos_vivos` ignoran las entidades muertas (`VIDA = 0`).

Comprueba cada ejercicio

Cada ejercicio trae casos de prueba (P1, P2…). Este challenge no tiene banco de pruebas automático, así que se comprueban así:

1. **En papel:** traza tu código a mano con cada caso. Los resultados esperados están en el **Anexo** del final: míralos solo después de trazar.
2. **En BGB:** pon un `ld b, b` justo antes del `call` a tu rutina (Options → Exceptions → «break on ld b,b»). Cuando pare, escribe en MEM y en los registros los datos del caso, pulsa F8/F7 hasta el `ret` y compara con el anexo. Con `ln -s obj/strongman-chicken.sym` BGB muestra los nombres de las etiquetas.
3. **En el juego:** cada ejercicio dice qué deberías ver «si funciona». Los stubs provisionales permiten compilar y jugar aunque falten piezas.

VBLANK es corto

Todo lo que escribe en VRAM u OAM durante el juego (ej. 2, 4 y 6b) se ejecuta dentro de `sys_render_update`, justo después de `wait_vblank`. Dura unos 1.140 ciclos de máquina: el código debe ser directo, sin esperas. Lo que escribe con el LCD apagado (ej. 1, 3 y 10c) no tiene límite.

CHALLENGE

Paso 0 · Código común

Este código es común para todos los ejercicios
Una persona lo añade al proyecto (unos 20 minutos) **antes** de empezar y lo sube. Vuestras soluciones deben usarlo tal cual. Junto con él van los **stubs**: cada rutina de ejercicio vacía con un resultado provisional, para que el proyecto compile desde el primer momento (en cada ejercicio veréis el stub). Cambia respecto a la versión actual: `rP1` (sin ella `engine/sys_input.asm` no compila), sprites activados en `LCDC_CONFIG`, índices de tiles nuevos (`TILE_NUM_0` deja de coincidir con el camino), `BOARD_ROWS = N_CARRILES`, `CosteStrongmen` en lugar de `COSTE_STRONG`, la llamada a `input_init` y el contador de frames.

engine/hardware.inc

```
;;=============================================================
;; Registros y direcciones del hardware que usa el juego
;;=============================================================
IF !DEF(HARDWARE_INC)
DEF HARDWARE_INC = 1

DEF rP1   = $FF00              ;; Joypad
DEF rLCDC = $FF40
DEF rLY   = $FF44
DEF rBGP  = $FF47
DEF rOBP0 = $FF48              ;; Paleta de los sprites

DEF LCDC_ON_BIT = 7
DEF VBLANK_LINE = 144

DEF VRAM_TILES = $8000
DEF TILE_TAM   = 16             ;; Bytes por tile

DEF BG_MAP         = $9800      ;; Tilemap del fondo
DEF PANTALLA_ANCHO = 20         ;; Tiles visibles por fila
DEF PANTALLA_ALTO  = 18         ;; Filas visibles
DEF TILEMAP_ANCHO  = 32         ;; Tiles por fila en el tilemap

DEF LCDC_CONFIG = %10010011     ;; LCD on, tiles en $8000, BG on, sprites 8x8 on
DEF BGP_CONFIG  = %11100100     ;; 3=negro, 2=gris oscuro, 1=gris claro, 0=blanco
DEF OBP0_CONFIG = %11100100     ;; Igual, pero en los sprites el color 0 es transparente

DEF OAM_RAM     = $FE00             ;; 40 sprites de 4 bytes: Y, X, TILE, ATTR
DEF OAM_TAM = 40 * 4
DEF OAM_Y_OFFSET = 16           ;; Y de la OAM = Y de la pantalla + 16
DEF OAM_X_OFFSET = 8            ;; X de la OAM = X de la pantalla + 8
DEF ATTR_XFLIP = %00100000      ;; Sprite volteado en horizontal
DEF ATTR_YFLIP = %01000000      ;; Sprite volteado en vertical

ENDC
```

gfx/gfx.inc

```
;;=============================================================
;; Graficos: indices en VRAM de los tiles (cargados en gfx_init)
;;=============================================================
IF !DEF(GFX_INC)
DEF GFX_INC = 1

DEF TILE_VACIO      = 0
DEF TILE_CASILLA    = 1  ;; 1..4  Casilla placeholder (sup-izq, sup-der, inf-izq, inf-der)
DEF TILE_CAMINO     = 5  ;;       Borde de arriba del carril
DEF TILE_CAMINO_INV = 6  ;;       Borde de abajo del carril
DEF TILE_POLLO      = 7  ;; 7..10 Pollo 2x2 (sup-izq, sup-der, inf-izq, inf-der)
DEF TILE_STRONG     = 11 ;; 11..14 Strongman 2x2 (mismo orden)
DEF TILE_CURSOR     = 15 ;;       Esquina sup-izq del cursor (las otras 3: volteada)
DEF TILE_NUM_0      = 16 ;; 16..25 Cifras 0..9

ENDC
```

man/man_entity.inc

```
;;===============================================================
;; Gestor de entidades: estructura y componentes
;;===============================================================
IF !DEF(MAN_ENTITY_INC)
DEF MAN_ENTITY_INC = 1
;;---------------------------------------------------------------
;; Entidades: array de MAX_ENTITIES entidades de SIZEOF_ENTITY bytes
;; Pollos (enemigos) y strongmen (torres) comparten el mismo array:
;; los distingue E_KIND
;;---------------------------------------------------------------
DEF MAX_ENTITIES = 40
RSRESET
DEF E_CMP         RB 1      ;; 0  Componentes de la entidad (bitflags). 0 = libre
DEF E_KIND        RB 1      ;; 1  Que es: KIND_CHICKEN, KIND_STRONG
DEF E_ESTADO      RB 1      ;; 2  Bit 7: se ha movido este frame (ESTADO_MOVIDO).
                            ;;    Bits 0-6: frame actual de la animacion
DEF E_VIDA        RB 1      ;; 3  Vida
DEF E_DANO        RB 1      ;; 4  Daño
DEF E_POS         RB 1      ;; 5  Casilla 2x2 (normalizada): %ffffcccc
                            ;;    ffff = fila (0-8), cccc = columna (0-9)
DEF SIZEOF_ENTITY RB 0      ;; = 6
DEF ENTITY_ARRAY_SIZE = MAX_ENTITIES * SIZEOF_ENTITY

;; El array esta alineado (ALIGN[8]) y cabe en 256 bytes:
;; pasar de una entidad a otra (o de un campo a otro) solo cambia L
ASSERT ENTITY_ARRAY_SIZE <= 256

;; Bits de CMP. CMP = 0 → la entidad esta LIBRE
DEF CMP_RENDER_BIT = 0
DEF CMP_MOVE_BIT   = 1
DEF CMP_ALIVE_BIT  = 7
;; Mascaras de CMP
DEF CMP_RENDER     = 1 << CMP_RENDER_BIT    ;; Se dibuja en su casilla (E_POS)
DEF CMP_MOVE       = 1 << CMP_MOVE_BIT      ;; Avanza por el carril (los pollos)
DEF CMP_ALIVE      = 1 << CMP_ALIVE_BIT

;; E_ESTADO
DEF ESTADO_MOVIDO_BIT = 7   ;; La pone sys_movement y la quita sys_render (borra la casilla vieja)
DEF ESTADO_MOVIDO     = 1 << ESTADO_MOVIDO_BIT

;; Una entidad con VIDA = 0 esta MUERTA: ya no hace nada.
;; sys_render borra su casilla y la libera (ver game_matar)

;; KIND
DEF KIND_CHICKEN = 1
DEF KIND_STRONG  = 2

ENDC
```

scn/scn_game.inc

```
;;===============================================================
;; Escena del juego: constantes
;;===============================================================
IF !DEF(SCN_GAME_INC)
DEF SCN_GAME_INC = 1

;;---------------------------------------------------------------
;; Strongmen (torres)
;;---------------------------------------------------------------
;; Numero de frames totales que suman las animaciones del strongman
DEF N_FRAMES_STRONG = 4

;; Numero de tipos diferentes de strongmen que hay en el juego
DEF N_TIPOS_STRONG = 6


;;---------------------------------------------------------------
;; Pollos (enemigos)
;;---------------------------------------------------------------
DEF CHICKEN_A = 0
DEF CHICKEN_B = 1
DEF CHICKEN_C = 2
```

```
23
24  DEF PRESET_SIZE = 2  ;; Bytes por cada tipo en la tabla chickenPresetsTable
25
26  DEF COL_ENTRADA      = 9           ;; Columna de casilla por la que entran los pollos
27  DEF MOVE_MASK        = %00111111   ;; Los pollos avanzan 1 casilla cada 64 frames
28  DEF ATAQUE_MASK      = %00001111   ;; Pollos y strongmen se pegan cada 16 frames
29  DEF RECOMPENSA_POLLO = 2           ;; Proteina por cada pollo derrotado
30
31  ;;-------------------------------------------------------------
32  ;; Mapa: carriles de 2 filas (en tiles)
33  ;;-------------------------------------------------------------
34  DEF FILA_CAMINO  = 10
35  DEF CAMINO_ANCHO = 20
36  DEF N_CARRILES   = 3
37  DEF FILA_CARRIL0 = FILA_CAMINO / 2 ;; Fila de CASILLA del primer carril (5)
38
39  ;;-------------------------------------------------------------
40  ;; Cursor del jugador
41  ;;-------------------------------------------------------------
42  DEF BOARD_COLS  = 7               ;; Columnas de casilla donde se colocan strongmen (0-6)
43  DEF BOARD_ROWS  = N_CARRILES      ;; Una fila del tablero por carril
44  DEF MENU_SLOTS  = 5
45  DEF FILA_MENU   = 0               ;; Fila de casilla del menu de compra (columnas 0..4)
46
47  ;;-------------------------------------------------------------
48  ;; Estado de la partida (wGameState)
49  ;;-------------------------------------------------------------
50  DEF GAME_JUGANDO = 0
51  DEF GAME_PERDIDO = 1              ;; Un pollo ha llegado a la columna 0 y quiere seguir
52  DEF GAME_GANADO  = 2              ;; Oleada terminada y no queda ningun pollo vivo
53  DEF POS_FINAL    = $44            ;; Casilla donde se dibuja el resultado (fila 4, col 4)
54
55  ASSERT MENU_SLOTS <= N_TIPOS_STRONG   ;; Cada hueco del menu es un tipo de strongman
56
57  ENDC
```

engine/frame.asm (nuevo)

```
1   ;;=============================================================
2   ;; Contador de frames
3   ;; Dato publico del motor: cualquiera lo lee, solo frame_tick lo escribe
4   ;;=============================================================
5   SECTION "Frame data", WRAM0
6   frame_counter:: ds 1            ;; Frames desde el principio (da la vuelta)
7
8   SECTION "Engine: frame", ROM0
9
10  ;; Usa: a
11  frame_init::
12      xor a
13      ld [frame_counter], a
14      ret
15
16  ;; Un frame mas. Lo llama el bucle principal una vez por frame
17  ;; Usa: hl
18  frame_tick::
19      ld hl, frame_counter
20      inc [hl]
21      ret
```

sys/sys_render.asm: código común (sustituye a sys_render_one y sys_render_update)

```
1   ;;-------------------------------------------------------------
2   ;; CODIGO COMUN
3   ;;-------------------------------------------------------------
4
5   ;; Primer tile de cada KIND (indice: E_KIND)
6   TilesKind:
7       db TILE_VACIO               ;; 0 (no se usa)
8       db TILE_POLLO               ;; KIND_CHICKEN
9       db TILE_STRONG              ;; KIND_STRONG
10
11  ;; Direccion del tilemap de la esquina sup-izq de una casilla
12  ;; Usa: a, bc, hl
```

```
;; Parametros entrada: a-posicion %ffffcccc
;; Parametros salida: hl-direccion = BG_MAP + fila * 64 + columna * 2
sys_render_pos_dir::
    ld b, a                     ;; \ C = columna * 2 (tiles)
    and $0F                     ;; |
    add a                       ;; |
    ld c, a                     ;; /
    ld a, b                     ;; \ HL = fila * 16
    and $F0                     ;; |
    ld l, a                     ;; |
    ld h, 0                     ;; /
    add hl, hl                  ;; \ HL = fila * 64 = fila * 2 * TILEMAP_ANCHO
    add hl, hl                  ;; /
    ld b, 0                     ;; \ HL += columna * 2
    add hl, bc                  ;; /
    ld bc, BG_MAP               ;; \ HL += inicio del tilemap
    add hl, bc                  ;; /
    ret

;; Dibuja una casilla 2x2 con 4 tiles consecutivos (sup-izq, sup-der, inf-izq, inf-der)
;; Requiere LCD apagado o estar en VBLANK
;; Usa: a, bc, hl
;; Parametros entrada: hl-dir de la esquina sup-izq, a-primer tile
;; Parametros salida: -
sys_render_casilla::
    ld [hl+], a                 ;; (x,   y)   sup-izq
    inc a
    ld [hl], a                  ;; (x+1, y)   sup-der
    ld bc, TILEMAP_ANCHO - 1
    add hl, bc                  ;; baja una fila y vuelve a la columna x
    inc a
    ld [hl+], a                 ;; (x,   y+1) inf-izq
    inc a
    ld [hl], a                  ;; (x+1, y+1) inf-der
    ret

;; Dibuja el fondo de una casilla de carril (borra lo que hubiera)
;; Requiere LCD apagado o estar en VBLANK
;; Usa: a, bc, hl
;; Parametros entrada: hl-dir de la esquina sup-izq
;; Parametros salida: -
sys_render_fondo::
    ld a, TILE_CAMINO
    ld [hl+], a
    ld [hl], a
    ld bc, TILEMAP_ANCHO - 1
    add hl, bc
    ld a, TILE_CAMINO_INV
    ld [hl+], a
    ld [hl], a
    ret

;; Espera a VBLANK y dibuja: cursor, HUD y TODAS las entidades con RENDER
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
sys_render_update::
    call wait_vblank
    call sys_cursor_update      ;; Ej. 4 (OAM: solo en VBLANK)
    call hud_update             ;; Ej. 6b
    ld hl, sys_render_one
    jp man_entity_for_each
```

scn/scn_game_rules.asm: tablas y estado (sustituye a las dos tablas del principio)

```
VidaStrongmen: db 10, 20, 30, 40, 50, 60
DanoStrongmen: db 1, 2, 3, 4, 5, 6
CosteStrongmen: db 2, 3, 4, 5, 6, 7

SECTION "Game state", WRAM0
wGameState:: ds 1               ;; GAME_JUGANDO, GAME_PERDIDO o GAME_GANADO

SECTION "Game rules (2)", ROM0
```

scn/scn_game_rules.asm: código común (al final; game_check_end se va a scn_game_end.asm)

```
;;-----------------------------------------------------------------
;; CODIGO COMUN
;;-----------------------------------------------------------------

;; Empieza la partida
;; Usa: a
game_estado_init::
    ld a, GAME_JUGANDO
    ld [wGameState], a
    ret

;; Devuelve en a el coste del tipo a
;; Usa: a, bc, hl
;; Parametros entrada: a-tipo strongman
;; Parametros salida: a-coste
game_strong_coste::
    ld hl, CosteStrongmen
    jr game_leer_tabla

;; Mata una entidad: VIDA = 0 y solo le queda RENDER (deja de moverse y de pegar).
;; sys_render borra su casilla y la libera
;; Usa: a
;; Parametros entrada: hl-entidad
;; Parametros salida: hl-entidad
game_matar::
    ld [hl], CMP_ALIVE | CMP_RENDER
    ld a, l                     ;; \ entity.VIDA = 0
    add E_VIDA                  ;; |
    ld l, a                     ;; |
    ld [hl], 0                  ;; /
    sub E_VIDA                  ;; \ HL = entidad otra vez
    ld l, a                     ;; /
    ret
```

scn/scn_game_oleada.asm (nuevo): datos de la oleada

```
;;=================================================================
;; Escena del juego: la oleada de pollos
;; Una oleada es una lista de entradas de 3 bytes:
;;   db espera, tipo, carril
;; espera = frames hasta que sale ese pollo (1-255), contados desde el anterior.
;; Una espera = 0 marca el final de la oleada.
;;=================================================================
INCLUDE "man/man_entity.inc"
INCLUDE "scn/scn_game.inc"

SECTION "Oleada data", WRAM0
wOleadaPtr::    ds 2            ;; Direccion del TIPO de la entrada actual
wOleadaEspera:: ds 1            ;; Frames que faltan para que salga
wOleadaFin::    ds 1            ;; 1 = ya no quedan pollos por salir

SECTION "Oleada", ROM0

;; La oleada del nivel (CODIGO COMUN)
Oleada1::
    db 180, CHICKEN_A, 1
    db  90, CHICKEN_A, 0
    db  90, CHICKEN_B, 2
    db 120, CHICKEN_A, 1
    db  60, CHICKEN_C, 0
    db  60, CHICKEN_B, 2
    db 0
```

scn/scn_game.asm

```
;;=================================================================
;; Escena: el juego
;; Su init y su update. El update solo decide el ORDEN de los sistemas (sys/)
;;=================================================================
INCLUDE "man/man_entity.inc"
INCLUDE "scn/scn_game.inc"

```

```
SECTION "Scene: game", ROM0

;; Inicializa la escena: sin entidades, carriles, cursor y proteina
;; Requiere LCD apagado
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
scene_game_init::
    call man_entity_init        ;; \ Sin entidades,
    call sys_render_init        ;; | la pantalla con los carriles,
    call sys_cursor_init        ;; | el menu y la OAM vacia,
    call sys_input_init         ;; | el cursor al principio,
    call moneda_init            ;; | la proteina inicial,
    call game_estado_init       ;; | la partida empezando
    ld hl, Oleada1              ;; | y la oleada preparada
    jp game_oleada_init         ;; /

;; Actualiza la escena (una vez por ciclo)
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
scene_game_update::
    call sys_input_update       ;; Actualizar el input del usuario en el modelo
    call game_oleada_update     ;; Salen pollos
    call sys_movement_update    ;; Los pollos avanzan
    call sys_combat_update      ;; Pollos y strongmen se pegan
    jp sys_render_update        ;; Renderizacion (espera a VBLANK)
```

main.asm (desde la línea 17)

```
;; Las entidades (pollos y strongmen) estan en man/man_entity.asm

SECTION "Entry point", ROM0[$150]

;; Inicializa la consola: graficos con el LCD apagado y la escena del juego
;; Usa: a, bc, de, hl
init_console::
   call screen_off         ;; \ Con el LCD apagado:
   call input_init         ;; | joypad,
   call frame_init         ;; | contador de frames,
   call gfx_init           ;; | tiles y paletas
   call scene_game_init    ;; / y la escena del juego
   jp screen_on

main::
   call init_console ;; Inicialización del programa

   ;; Por cada ciclo
   game_mainloop:
      call input_update      ;; Lee el joypad -> wPad*
      call frame_tick        ;; Un frame mas
      call scene_game_update ;; Input, simulación y renderización
      call game_check_end    ;; Ha acabado la partida? (Z) -> game_end
   jr nz, game_mainloop

   game_end:
   call game_pantalla_final ;; Has ganado o has perdido

   di     ;; Disable Interrupts
   halt   ;; Halt the CPU (stop procesing here)
```

> **Ficheros nuevos con solo stubs**
> `sys/sys_cursor.asm`, `sys/sys_combat.asm` y `scn/scn_game_end.asm` se crean con la cabecera (`INCLUDE` y `SECTION`) y los stubs que aparecen en cada ejercicio. `sys/sys_simulation.asm` ya no se llama y `game_check_end` se borra de `scn_game_rules.asm`, porque pasa a ser el Ejercicio 10b.

Contratos de lo que ya está hecho

Contrato: **Entrada** · **Salida** · **Modifica** (registros que puede cambiar).

Rutina	Qué hace	Entrada → Salida	Modifica
`man_entity_alloc`	Reserva una entidad libre	A = componentes → Z, HL = entidad · NZ: lleno	A, BC, HL
`man_entity_free`	Libera una entidad	DE = entidad	A
`man_entity_for_each`	Llama a la rutina por cada entidad viva	HL = rutina (recibe DE = entidad)	todos
`memcpy`	Copia BC bytes	HL = origen, DE = destino, BC > 0	A, BC, DE, HL
`memset`	Rellena BC bytes con D	HL = destino, BC > 0, D = valor	A, BC, HL
`wait_vblank`	Espera a la línea 144	— (LCD encendido)	A
`screen_off / screen_on`	Apaga (en VBLANK) / enciende el LCD	—	A, HL / A
`sys_render_init`	Borra la pantalla y dibuja los carriles	— (LCD apagado)	A, BC, DE, HL
`sys_render_pos_dir`	Dirección del tilemap de una casilla	A = POS → HL	A, BC, HL
`sys_render_casilla`	Dibuja 4 tiles consecutivos en 2×2	HL = dirección, A = primer tile	A, BC, HL
`sys_render_fondo`	Dibuja una casilla de carril vacía	HL = dirección	A, BC, HL
`game_strong_colocar`	Crea un strongman	A = tipo, B = POS → Z · NZ: lleno	A, BC, DE, HL
`game_chicken_spawn`	Crea un pollo (RENDER | MOVE)	A = tipo, B = POS → Z · NZ: lleno	A, BC, DE, HL
`game_strong_coste`	Coste de un tipo de strongman	A = tipo → A = coste	A, BC, HL
`game_matar`	VIDA = 0, CMP = ALIVE | RENDER	HL = entidad → HL = entidad	A
`moneda_sumar`	Proteína += B (máx. 99)	B	A
`moneda_restar`	Proteína -= B si hay saldo	B → NC: restado · C: no hay saldo	A
`hud_dibujar_numero`	Dibuja 00-99 con 2 tiles (espera VBLANK)	A = número, B = columna, C = fila	A, BC, DE, HL
`hud_dibujar_moneda`	Dibuja wProtein en su sitio del HUD	—	A, BC, DE, HL

Dato	Qué es	Quién lo escribe
`entity_array`	Las entidades (alineado, 240 bytes)	man_entity, los sistemas
`frame_counter`	Frames desde el principio (da la vuelta)	frame_tick
`wPadUp … wPadB`	1 = botón recién pulsado este frame	input_update
`wCursorState`	0 = tablero, 1 = menú	sys_input
`wCursorRow, wCursorCol`	Casilla del tablero bajo el cursor (fila 0-2, columna 0-6)	sys_input
`wSelectedRow, wSelectedCol`	Casilla elegida con A antes de abrir el menú	sys_input
`wMenuSlot`	Hueco del menú (0-4) = tipo de strongman	sys_input
`wProtein`	Saldo de proteína	moneda_*
`wGameState`	GAME_JUGANDO, GAME_PERDIDO o GAME_GANADO	Ej. 8, Ej. 10b
`wOleadaPtr, wOleadaEspera, wOleadaFin`	Estado de la oleada	Ej. 7

Ejercicio 1

Implementa los gráficos: Tiles_pollo, Tiles_strong, Tile_cursor, Tiles_numeros y gfx_init

Apuntes: «Cómo crear gráficos a mano (roca de Pokémon)» y `gfx_init` del Challenge 5. Faltan 19 tiles:

Pollo (7-10) y **strongman** (11-14): dibujos de 16 × 16 píxeles en 4 tiles, en el mismo orden que `Casilla_placeholder` (sup-izq, sup-der, inf-izq, inf-der). El pollo mira a la **izquierda**, que es hacia donde camina. **Cursor** (15): solo la esquina de arriba a la izquierda, porque las otras tres son el mismo tile volteado. **Cifras** (16-25): del 0 al 9, para el HUD.

Después, `gfx_init` debe copiarlos a VRAM en su índice (ver `gfx.inc`) y poner la paleta de los sprites: `rOBP0 = OBP0_CONFIG`.

Pasos (intentadlo antes sin mirarlos)

1. Dibujad en papel una cuadrícula de 16 × 16 por cada figura y una de 8 × 8 para el cursor y para cada cifra. Usad 4 colores: 0 blanco, 1 gris claro, 2 gris oscuro, 3 negro.
2. Cada fila de 8 píxeles son 2 bytes. El primero tiene el bit **bajo** de cada píxel y el segundo el bit **alto** (el píxel de la izquierda es el bit 7).
3. Escribid los `db` bajo cada etiqueta, con `.fin:` al final (como `Casilla_placeholder`).
4. En `gfx_init`, un `memcpy` por bloque: `HL` = datos, `DE = VRAM_TILES + TILE_xxx * TILE_TAM`, `BC = .fin - inicio`.
5. `ld a, OBP0_CONFIG` / `ld [rOBP0], a`.

Consejos

- En los sprites el color 0 es **transparente**: el cursor debe ser una esquina fina y el resto a 0, para que se vea lo que hay debajo.
- Dejad la última columna de cada cifra en blanco: así no se pegan dos cifras seguidas.
- `memcpy` con `BC = 0` copia 65.536 bytes y borra media memoria. No la llaméis con una etiqueta que todavía no tiene datos.
- Los fondos de los carriles son grises (`TILE_CAMINO`). Un pollo con borde negro se distingue mejor.

Si funciona

- Con el ej. 3, el menú muestra 5 strongmen arriba. Con el 2, los pollos y los strongmen se ven en los carriles. Con el 4, el cursor se ve en 4 esquinas. Con el 6b, el HUD muestra los números.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, codificar una fila.** Píxeles de izquierda a derecha: `0 3 3 1 0 0 2 0`. ¿Qué dos bytes son?
- **P2, direcciones.** ¿En qué dirección de VRAM empieza la cifra 0? ¿Cuál es el último byte de la cifra 9?

gfx/gfx.asm (desde Tiles_pollo)

```
Tiles_pollo:     ;; ●●4 tiles: 7..10 (sup-izq, sup-der, inf-izq, inf-der)●●
.fin:

Tiles_strong:    ;; ●●4 tiles: 11..14●●
.fin:

Tile_cursor:     ;; ●●1 tile: 15 (esquina sup-izq)●●
.fin:

Tiles_numeros:   ;; ●●10 tiles: 16..25 (cifras 0..9)●●
.fin:

SECTION "Engine: graphics", ROM0

;; Carga los tiles en VRAM y la paleta
```

```
;; Requiere LCD apagado
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
gfx_init::
    ;; Tile 0 en blanco (es el que se usa para borrar)
    ld hl, VRAM_TILES + TILE_VACIO * TILE_TAM
    ld bc, TILE_TAM
    ld d, 0
    call memset

    ;; Casilla 2x2 en los tiles 1..4
    ld hl, Casilla_placeholder
    ld de, VRAM_TILES + TILE_CASILLA * TILE_TAM
    ld bc, Casilla_placeholder.fin - Casilla_placeholder
    call memcpy

    ;; Define camino tile 5
    ld hl, Tile_camino
    ld de, VRAM_TILES + TILE_CAMINO * TILE_TAM
    ld bc, Tile_camino.fin - Tile_camino
    call memcpy

    ;; Define camino_inv tile 6
    ld hl, Tile_camino_inv
    ld de, VRAM_TILES + TILE_CAMINO_INV * TILE_TAM
    ld bc, Tile_camino_inv.fin - Tile_camino_inv
    call memcpy

    ;; ●●TU CÓDIGO: carga Tiles_pollo, Tiles_strong, Tile_cursor y Tiles_numeros●●

    ld a, BGP_CONFIG
    ld [rBGP], a
    ;; ●●TU CÓDIGO: paleta de los sprites●●
    ret
```

Ejercicio 2

Implementa sys_render_one

Apuntes: S05.2 («Un sistema»: el sistema solo sabe procesar **una** entidad) y `sys_prerender_one` (Ch. 5 ej. 6). Dibuja una entidad con `RENDER`. `DE` = entidad.

1. Si está **muerta** (`VIDA = 0`): dibuja el fondo en su casilla (`sys_render_fondo`) y la libera (`man_entity_free`). Nada más.
2. Si tiene la marca `ESTADO_MOVIDO`: quita la marca y dibuja el fondo en la casilla de su **derecha** (`E_POS + 1`), que es de donde viene.
3. Dibuja su casilla con el primer tile de su `KIND`: `TilesKind[KIND]` y `sys_render_casilla`.

Pasos (intentadlo antes sin mirarlos)

1. Filtro: `ld a, [de]` / `and CMP_RENDER` / `ret z`.
2. El array está alineado: la dirección de un campo es `H = D`, `L = E + E_CAMPO`.
3. Muerta: `A = POS`, `sys_render_pos_dir`, `sys_render_fondo` y `jp man_entity_free` (`DE` sigue siendo la entidad).
4. Marca: `bit ESTADO_MOVIDO_BIT, [hl]`, `res ESTADO_MOVIDO_BIT, [hl]`, `A = POS + 1`, `pos_dir` y `fondo`.
5. Dibujo: `C = KIND`, `HL = sys_render_pos_dir(POS)`, `A = TilesKind[C]` (con `B = 0` y `add hl, bc`, guardando antes el `HL` de la casilla) y `jp sys_render_casilla`.

Consejos

- `sys_render_pos_dir` y `sys_render_casilla` modifican `BC`. Guardad en la pila lo que necesitéis después.
- No toquéis `DE` en el caso «muerta»: `man_entity_free` lo necesita.
- Si dos pollos se mueven en el mismo frame, uno puede borrar al otro un frame. No pasa nada: `sys_render` vuelve a dibujar todo en el frame siguiente.

Si funciona

- Los strongmen aparecen al comprarlos, los pollos avanzan sin dejar rastro y, al morir, pollo o strongman desaparecen y el carril queda limpio.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, un pollo que se ha movido.** Pollo vivo (`VIDA = 5`) en `E_POS = $57` con `E_ESTADO = $80`. El tilemap está a 0.
- **P2, un strongman muerto.** Strongman con `VIDA = 0` en `E_POS = $63`.

sys/sys_render.asm: sys_render_one

```
;;-----------------------------------------------------------------
;; EJERCICIO 2
;;-----------------------------------------------------------------
;; Dibuja UNA entidad con RENDER:
;;  1. Si esta muerta (VIDA = 0): dibuja el fondo en su casilla y la libera.
;;  2. Si se ha movido (ESTADO_MOVIDO): quita la marca y dibuja el fondo en la
;;     casilla de su derecha (de donde viene).
;;  3. Dibuja su casilla con el primer tile de su KIND (tabla TilesKind).
;; Requiere estar en VBLANK
;; Usa: a, bc, de, hl
;; Parametros entrada: de-entidad
;; Parametros salida: -
sys_render_one::
    ;; ●●TU CÓDIGO●●
    ret
```

Ejercicio 3

Implementa sys_cursor_init

Apuntes: OAM y sprites del Challenge 5 (`oam_clear`) y bucles (S01.3). Con el LCD apagado, deja la OAM sin sprites y dibuja el menú de compra.

1. Pon a 0 los `OAM_TAM` bytes de `OAM_RAM`: un sprite con `Y = 0` no se ve.
2. Dibuja un strongman (`TILE_STRONG`) en cada hueco del menú: casillas `(FILA_MENU, 0)` … `(FILA_MENU, MENU_SLOTS - 1)`.

Pasos (intentadlo antes sin mirarlos)

1. `memset` con `HL = OAM_RAM`, `BC = OAM_TAM`, `D = 0`.
2. Un registro con la posición del hueco: empieza en `FILA_MENU << 4`.
3. En cada vuelta: `A = posición`, `sys_render_pos_dir`, `A = TILE_STRONG`, `sys_render_casilla`.
4. Siguiente hueco con `inc`. Se termina cuando la posición llega a `(FILA_MENU << 4) | MENU_SLOTS`.

Consejos

- `memset` y las `sys_render_*` modifican `A`, `BC` y `HL`. El contador debe ir en `D` o `E`.
- El LCD está apagado (lo llama `scene_game_init`): no hace falta esperar a VBLANK.

Si funciona

- Al empezar se ven 5 strongmen en la fila de arriba y no aparece ningún sprite basura.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, la OAM.** Los 160 bytes de la OAM valen `$FF`. ¿Cómo quedan?
- **P2, el menú.** El tilemap está a 0. ¿Qué tiles hay en `$9800-$980A` y en `$9820-$982A`?

sys/sys_cursor.asm: sys_cursor_init

```
;;-------------------------------------------------------------
;; EJERCICIO 3
;;-------------------------------------------------------------
;; Deja la OAM sin sprites (toda a 0) y dibuja el menu de compra:
;; un strongman (TILE_STRONG) en cada hueco, casillas (FILA_MENU, 0..MENU_SLOTS-1)
;; Requiere LCD apagado
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
sys_cursor_init::
    ;; ◉◉TU CÓDIGO◉◉
    ret
```

Ejercicio 4

Implementa sys_cursor_update

Apuntes: sprites y coordenadas OAM (Ch. 5 y Ch. 6 ej. 8). El cursor son 4 sprites (0-3) en las 4 esquinas de una casilla, con el mismo tile volteado. Se llama en VBLANK, justo después de `wait_vblank`.

Casilla: en el tablero (`wCursorState = 0`), fila `FILA_CARRIL0 + wCursorRow` y columna `wCursorCol`. En el menú (`wCursorState = 1`), fila `FILA_MENU` y columna `wMenuSlot`.
Coordenadas OAM: `y = fila × 16 + OAM_Y_OFFSET` y `x = columna × 16 + OAM_X_OFFSET`. Los 4 sprites (Y, X, TILE, ATTR): `(y, x, T, 0)`, `(y, x+8, T, ATTR_XFLIP)`, `(y+8, x, T, ATTR_YFLIP)`, `(y+8, x+8, T, ATTR_XFLIP | ATTR_YFLIP)`, con `T = TILE_CURSOR`.

Pasos (intentadlo antes sin mirarlos)

1. Según `wCursorState`, `B = fila` y `C = columna`.
2. `B = y`: `ld a, b` / `swap a` (× 16) / `add OAM_Y_OFFSET`. Lo mismo para `C = x`.
3. `HL = OAM_RAM` y 16 escrituras con `ld [hl+], a`.

Consejos

- `swap a` multiplica por 16 un número de 0 a 15 (el nibble alto queda a 0).
- La OAM solo se puede escribir en VBLANK o con el LCD apagado. No la escribáis desde otro sitio.

Si funciona

- El cursor se mueve por los carriles con la cruceta. Con A salta a los huecos del menú y se mueve por ellos; con B vuelve al tablero.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, en el tablero.** `wCursorState = 0`, `wCursorRow = 1`, `wCursorCol = 3`. ¿Qué 16 bytes quedan en `$FE00-$FE0F`?
- **P2, en el menú.** `wCursorState = 1`, `wMenuSlot = 4`.

sys/sys_cursor.asm: sys_cursor_update

```
;;-----------------------------------------------------------------
;; EJERCICIO 4
;;-----------------------------------------------------------------
;; Coloca el cursor (sprites 0..3 de la OAM) en las 4 esquinas de su casilla:
;;   Tablero (wCursorState = 0): fila FILA_CARRIL0 + wCursorRow, columna wCursorCol
;;   Menu    (wCursorState = 1): fila FILA_MENU, columna wMenuSlot
;; Esquina   Y      X      TILE         ATTR
;; sup-izq   y      x      TILE_CURSOR  0
;; sup-der   y      x+8    TILE_CURSOR  ATTR_XFLIP
;; inf-izq   y+8    x      TILE_CURSOR  ATTR_YFLIP
;; inf-der   y+8    x+8    TILE_CURSOR  ATTR_XFLIP | ATTR_YFLIP
;; con y = fila * 16 + OAM_Y_OFFSET, x = columna * 16 + OAM_X_OFFSET
;; Requiere estar en VBLANK
;; Usa: a, bc, hl
;; Parametros entrada: -
;; Parametros salida: -
sys_cursor_update::
    ;; ●●TU CÓDIGO●●
    ret
```

Ejercicio 5

Implementa game_entidad_en

Apuntes: S05.2 y `man_entity_alloc` (Ch. 5 ej. 1). Busca una entidad **viva** (`CMP_ALIVE` y `VIDA ≠ 0`) en la casilla `B`. La usan los ejercicios 6, 7, 8 y 9 para saber si una casilla está ocupada y quién hay.

Entrada: `B` = posición. **Salida:** `Z` y `HL` = la entidad (su primer byte, `CMP`), o `NZ` si no hay ninguna. **No** puede modificar `B` ni `DE`.

Pasos (intentadlo antes sin mirarlos)

1. Recorred el array como `man_entity_alloc`: `HL = entity_array`, un contador con `MAX_ENTITIES` y `add SIZEOF_ENTITY` sobre `L`.
2. Por cada entidad: si no está viva, la siguiente. Si su `VIDA` es 0, la siguiente. Si su `POS` es `B`, encontrada.
3. Al encontrarla, `HL` vuelve al principio de la entidad y devolvéis `Z` (`xor a`). Si se acaba el array, `NZ` (`or 1`).

Consejos

- No sirve `man_entity_for_each`: hay que parar en cuanto se encuentra y devolver `HL`.
- El contador va en `C`: `B` y `DE` son del que llama.
- Para mirar un campo cambiáis `L`. Llevad la cuenta de cuánto le habéis sumado para poder volver al principio de la entidad.

Si funciona

- No se puede comprar en una casilla ocupada (ej. 6). Los pollos no se pisan (ej. 8), no aparecen encima de otro (ej. 7) y encuentran al strongman (ej. 9).

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, ocupada.** Entidad 0: strongman en `$52`, `VIDA = 10`. Entidad 1: pollo en `$57`, `VIDA = 5`. El resto, libres. `B = $57`.
- **P2, solo hay un muerto.** Igual que P1, pero el strongman tiene `VIDA = 0` y `B = $52`.

scn/scn_game_rules.asm: game_entidad_en

```
;;-----------------------------------------------------------------
;; EJERCICIO 5
;;-----------------------------------------------------------------
;; Busca una entidad VIVA (CMP_ALIVE y VIDA != 0) que este en la casilla b
;; Usa: a, c, hl. NO modifica b ni de
;; Parametros entrada: b-posicion
;; Parametros salida: Z, hl-entidad encontrada
;;                    NZ, la casilla esta libre
game_entidad_en::
    ;; ◉◉TU CÓDIGO◉◉
    or 1                        ;; (provisional: siempre libre)
    ret
```

Ejercicio 6

Corrige game_comprar_strong (a) e implementa hud_update (b)

Apuntes: `man_entity_alloc` y `sys_physics_one` corregidas (Ch. 5 ej. 1 y 3) y el HUD (Ch. 6 ej. 4).

(a) `sys_input` llama a `game_comprar_strong` al pulsar A en el menú. Coloca un strongman de tipo `wMenuSlot` en la casilla elegida (fila del tablero `wSelectedRow`, que es la fila de casilla `FILA_CARRIL0 + wSelectedRow`, y columna `wSelectedCol`) y lo paga. Si la casilla está ocupada, si no hay saldo o si no quedan entidades, no hace nada y devuelve `C`.

(b) `hud_update` dibuja la proteína (`hud_dibujar_moneda`) y, en `(HUD_COSTE_COL, HUD_COSTE_FILA)`, el coste del strongman del hueco `wMenuSlot`.

Corregir
El código de (a) tiene **7 errores**: de lógica (E), instrucciones que no existen (N) y cosas que faltan (F). Encontradlos en papel, copiad el código en `game_comprar_strong` y corregidlo.

Pasos (intentadlo antes sin mirarlos)

1. (a) Comprobad el orden: primero la casilla, después el saldo y por último colocar. Solo se cobra si se ha colocado.
2. (a) Mirad el contrato de cada rutina a la que llama: qué registros modifica y cómo devuelve el resultado (Z/NZ o C/NC).
3. (b) `call hud_dibujar_moneda`, y después `A = game_strong_coste(wMenuSlot)`, `B` y `C` = columna y fila, y `jp hud_dibujar_numero`.

Consejos

- `ld r, [n16]` solo existe con `A`.
- La fila de casilla es `FILA_CARRIL0 + fila del tablero`, y se pasa al nibble alto **después** de sumar.
- `or a` deja el carry a 0. Para ponerlo a 1 está `scf`.
- `hud_dibujar_numero` ya espera a VBLANK y `hud_update` se llama dentro de VBLANK, así que no espera.

Si funciona

- Con A sobre una casilla libre, A otra vez en el menú: aparece el strongman, la proteína baja y el cursor vuelve al tablero. Si no hay saldo o la casilla está ocupada, el cursor se queda en el menú. El HUD muestra el coste del hueco elegido.

Casos de prueba
Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1 (a), se compra.** `wSelectedRow = 0`, `wSelectedCol = 2`, `wMenuSlot = 1`, `wProtein = 10` y ninguna entidad.
- **P2 (a), sin saldo.** Igual, pero `wProtein = 2`.
- **P3 (b).** `wProtein = 7`, `wMenuSlot = 1`. ¿Qué tiles quedan en `$9A20`, `$9A21`, `$9A32` y `$9A33`?

`Ejercicio 6 (a): game_comprar_strong, con errores`

```
;;-------------------------------------------------------------
;; EJERCICIO 6 (a)
;;-------------------------------------------------------------
;; Paga y coloca un strongman en la casilla elegida con el cursor
;; Tipo = wMenuSlot, casilla = (FILA_CARRIL0 + wSelectedRow, wSelectedCol)
;; No hace nada si la casilla esta ocupada o no hay saldo suficiente
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: NC, comprado. C, no se ha comprado
game_comprar_strong::
    ld a, [wSelectedRow]        ;; \ B = posicion = %ffffcccc
    swap a                      ;; |
```

```
    add FILA_CARRIL0            ;; |
    ld b, a                     ;; |
    ld a, [wSelectedCol]        ;; |
    or b                        ;; |
    ld b, a                     ;; /

    call game_entidad_en        ;; \ Ocupada?
    jr nz, .no                  ;; /

    ld a, [wMenuSlot]           ;; \ C = coste del tipo
    call game_strong_coste      ;; |
    ld c, a                     ;; /

    ld b, [wProtein]            ;; \ Hay saldo?
    cp c                        ;; |
    ret nc                      ;; / No: C

    push bc                     ;; \ Colocamos el strongman
    ld a, [wMenuSlot]           ;; |
    call game_strong_colocar    ;; |
    pop bc                      ;; |
    jr nz, .no                  ;; / No hay hueco en el array

    jp moneda_restar            ;; Pagamos: NC

    .no:
        or a
        ret
```

ui/hud.asm: hud_update

```
;;-----------------------------------------------------------------
;; EJERCICIO 6 (b)
;;-----------------------------------------------------------------
;; Dibuja el HUD: el saldo (wProtein) en (HUD_MONEDA_COL, HUD_MONEDA_FILA)
;; y el coste del strongman del hueco wMenuSlot en (HUD_COSTE_COL, HUD_COSTE_FILA)
;; Requiere estar en VBLANK
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
hud_update::
    ;; ◉◉TU CÓDIGO◉◉
    ret
```

Ejercicio 7

Implementa game_oleada_init (a) y game_oleada_update (b)

Apuntes: `game_enemies_spawn` (Ch. 6 ej. 5): crear entidades a partir de datos. Una oleada es una lista de entradas de 3 bytes, `db espera, tipo, carril`, que termina con una espera 0. La espera son los frames que pasan desde el pollo anterior.

(a) `game_oleada_init` (`HL` = oleada): si la primera espera es 0, `wOleadaFin = 1`. Si no, `wOleadaFin = 0`, `wOleadaEspera` = esa espera y `wOleadaPtr` apunta a su **tipo**.

(b) `game_oleada_update` (una vez por frame): si la oleada no ha terminado, `wOleadaEspera--`. Al llegar a 0:

1. `B` = casilla de entrada del carril: fila `FILA_CARRIL0 + carril`, columna `COL_ENTRADA`.
2. Si está ocupada (`game_entidad_en` da `Z`): `wOleadaEspera = 1` y `ret`. Se vuelve a intentar en el frame siguiente.
3. Si no: `game_chicken_spawn` con `A` = tipo y `B` = casilla. Avanza a la entrada siguiente: si su espera es 0, `wOleadaFin = 1`. Si no, `wOleadaEspera` = su espera y `wOleadaPtr` = su tipo.

Pasos (intentadlo antes sin mirarlos)

1. (a) `ld a, [hl+]` lee la espera y deja `HL` en el tipo. `wOleadaPtr` es un puntero: guardad `L` y luego `H`.
2. (b) `ld hl, wOleadaEspera` / `dec [hl]` / `ret nz`.
3. (b) `DE = wOleadaPtr`: `[de]` es el tipo y `[de+1]` el carril.
4. (b) `game_chicken_spawn` modifica `DE`: guardadlo en la pila.
5. (b) Tras crear el pollo, la entrada siguiente empieza 2 bytes después del tipo: su espera, y detrás su tipo.

Consejos

- `game_entidad_en` no modifica `B` ni `DE`, pero `game_chicken_spawn` sí.
- Guardad el puntero en el mismo orden en que lo leéis: primero el byte bajo.

Si funciona

- Salen 6 pollos, uno tras otro, por los 3 carriles. Si un pollo sigue en la casilla de entrada, el siguiente espera a que se mueva. Al salir el último, `wOleadaFin = 1`.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1 (a).** `game_oleada_init` con `HL = Oleada1`.
- **P2 (b), toca crear.** `wOleadaFin = 0`, `wOleadaEspera = 1`, `wOleadaPtr` apunta al tipo de la tercera entrada de `Oleada1` (`CHICKEN_B`, carril 2). No hay entidades.
- **P3 (b), aún no.** `wOleadaEspera = 5`.

scn/scn_game_oleada.asm: game_oleada_init

```
;;-----------------------------------------------------------------
;; EJERCICIO 7 (a)
;;-----------------------------------------------------------------
;; Prepara una oleada: si la primera espera es 0, wOleadaFin = 1.
;; Si no: wOleadaFin = 0, wOleadaEspera = primera espera y wOleadaPtr apunta a su tipo
;; Usa: a, hl
;; Parametros entrada: hl-oleada
;; Parametros salida: -
game_oleada_init::
    ;; ●●TU CÓDIGO●●
    ld a, 1                     ;; (provisional: oleada vacia)
    ld [wOleadaFin], a
    ret
```

scn/scn_game_oleada.asm: game_oleada_update

```
;;-------------------------------------------------------------
;; EJERCICIO 7 (b)
;;-------------------------------------------------------------
;; Una vez por frame. Si la oleada no ha terminado, descuenta la espera. Al llegar a 0:
;;  - Si la casilla de entrada (FILA_CARRIL0 + carril, COL_ENTRADA) esta ocupada,
;;    espera 1 frame mas (wOleadaEspera = 1) y lo vuelve a intentar.
;;  - Si no, crea el pollo y pasa a la siguiente entrada (o termina si su espera es 0)
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
game_oleada_update::
    ;; ◉◉TU CÓDIGO◉◉
    ret
```

Ejercicio 8

Implementa sys_movement_one (a) y sys_movement_update (b)

Apuntes: `sys_physics_one` y `sys_physics_update` (Ch. 5 ej. 3). Mueve los pollos (componente `MOVE`) una casilla a la izquierda. `DE` = entidad.

(a) 1. Si está en la columna 0, ha llegado a la casa: `wGameState = GAME_PERDIDO`.
2. Si la casilla de su izquierda (`POS - 1`) está ocupada, no se mueve (de eso se encarga el combate).
3. Si no: `POS = POS - 1` y marca `ESTADO_MOVIDO` en `E_ESTADO` (`sys_render` borrará la casilla vieja).
(b) Solo un frame de cada 64 (`frame_counter AND MOVE_MASK = 0`), llama a `sys_movement_one` para todas las entidades (`man_entity_for_each`).

Pasos (intentadlo antes sin mirarlos)

1. Filtro `CMP_MOVE` (como `sys_physics_one`).
2. `A = POS` / `and $0F` / `jr z, .casa`.
3. `B = POS - 1` / `call game_entidad_en` / `ret z`.
4. Volved a calcular la dirección de `E_POS` (`game_entidad_en` ha cambiado `HL`) y escribid `B`. Después `set ESTADO_MOVIDO_BIT, [hl]` sobre `E_ESTADO`.

Consejos

- `game_entidad_en` respeta `B` y `DE`: la nueva posición sigue en `B` y la entidad en `DE`.
- `dec` sobre la posición solo cambia la columna, porque la columna nunca es 0 en ese punto.

Si funciona

- Los pollos avanzan una casilla por segundo, más o menos. Se paran detrás de otro pollo o delante de un strongman. Si uno llega a la casa, la partida termina (con el ej. 10).

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, avanza.** Pollo en `$54`, `E_ESTADO = 0`, `$53` libre, `wGameState = 0`.
- **P2, llega a la casa.** Pollo en `$50`.
- **P3, bloqueado.** Pollo en `$54` y un strongman vivo en `$53`.

sys/sys_movement.asm: sys_movement_one

```
;;-----------------------------------------------------------------
;; EJERCICIO 8 (a)
;;-----------------------------------------------------------------
;; Mueve UNA entidad con MOVE una casilla a la izquierda:
;;  - Si esta en la columna 0: ha llegado a la casa -> wGameState = GAME_PERDIDO
;;  - Si la casilla de su izquierda esta ocupada, no se mueve (sys_combat se encarga)
;;  - Si no: POS - 1 y marca ESTADO_MOVIDO (sys_render borrara la casilla vieja)
;; Usa: a, bc, de, hl
;; Parametros entrada: de-entidad
;; Parametros salida: -
sys_movement_one::
    ;; ●●TU CÓDIGO●●
    ret
```

sys/sys_movement.asm: sys_movement_update

```
;; EJERCICIO 8 (b): mueve todas las entidades con MOVE, pero solo 1 de cada 64 frames
;; (cuando frame_counter AND MOVE_MASK = 0)
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
sys_movement_update::
    ;; ●●TU CÓDIGO●●
    ret
```

Ejercicio 9

Implementa sys_combat_one (a) y sys_combat_update (b)

Apuntes: `sys_collision_one` (Ch. 5 ej. 10): dos entidades que se tocan y lo que pasa. Un pollo (`MOVE`) con un strongman en la casilla de su izquierda: se pegan **los dos a la vez**. `DE` = entidad.

(a) 1. Solo pollos (`CMP_MOVE`). En la columna 0 no hay nada a la izquierda.
2. `game_entidad_en(POS - 1)`: si no hay nadie, o no es `KIND_STRONG` (es otro pollo), `ret`.
3. `strongman.VIDA -= pollo.DANO`. Si `VIDA ≤ DANO`: `game_matar` (strongman).
4. `pollo.VIDA -= strongman.DANO`. Si `VIDA ≤ DANO`: `game_matar` (pollo) y `moneda_sumar` con `B = RECOMPENSA_POLLO`.
(b) Solo un frame de cada 16 (`frame_counter AND ATAQUE_MASK = 0`), para todas las entidades.

Pasos (intentadlo antes sin mirarlos)

1. Tras `game_entidad_en`, `HL` = strongman y `DE` = pollo. Están en el mismo array, así que `H = D` y cada entidad se recuerda con su byte bajo: guardad el del strongman (`L`) en `C`.
2. Leed `pollo.DANO` en `B` y restadlo a `strongman.VIDA`. `sub` da `Z` si llega a 0 y `C` si se pasa: los dos casos son muerte.
3. Leed `strongman.DANO` (aunque acabe de morir: es a la vez) y restadlo a `pollo.VIDA`.
4. `game_matar` recibe `HL` = entidad y solo modifica `A`.

Consejos

- Si no escribís `VIDA` cuando muere, `game_matar` la deja a 0.
- El byte `KIND` es `E_KIND` = 1: `inc l` / `ld a, [hl]` / `dec l`.
- Los dos pierden vida en el mismo golpe. No comprobéis si el strongman ha muerto antes de pegar al pollo.

Si funciona

- Cuando un pollo llega a un strongman, los dos se quedan parados y se pegan. El que muere desaparece. Si muere el pollo, la proteína sube 2. Si muere el strongman, el pollo sigue andando.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, se pegan.** Pollo en `$54`, `VIDA = 10`, `DANO = 1`. Strongman en `$53`, `VIDA = 20`, `DANO = 2`. `wProtein = 10`.
- **P2, muere el pollo.** Igual, pero el pollo tiene `VIDA = 2`.

sys/sys_combat.asm: sys_combat_one

```
;;-----------------------------------------------------------------
;; EJERCICIO 9 (a)
;;-----------------------------------------------------------------
;; Si DE es un pollo (MOVE) y en la casilla de su izquierda hay un strongman:
;;   strongman.VIDA -= pollo.DANO   (si VIDA <= DANO: game_matar)
;;   pollo.VIDA     -= strongman.DANO (si VIDA <= DANO: game_matar y
;;                                     moneda_sumar RECOMPENSA_POLLO)
;; Usa: a, bc, de, hl
;; Parametros entrada: de-entidad
;; Parametros salida: -
sys_combat_one::
    ;; ●●TU CÓDIGO●●
    ret
```

sys/sys_combat.asm: sys_combat_update

```
;; EJERCICIO 9 (b): combate de todos los pollos, pero solo 1 de cada 16 frames
;; (cuando frame_counter AND ATAQUE_MASK = 0)
;; Usa: a, bc, de, hl
```

```
;; Parametros entrada: -
;; Parametros salida: -
sys_combat_update::
    ;; ◉◉TU CÓDIGO◉◉
    ret
```

Ejercicio 10

Implementa game_pollos_vivos (a), game_check_end (b) y game_pantalla_final (c)

Apuntes: escenas y cambio de escena (Ch. 5 ej. 11-12) y `is_player_gone` (Ch. 6).

(a) `game_pollos_vivos`: `A` = número de entidades con `CMP_ALIVE`, `KIND_CHICKEN` y `VIDA ≠ 0`.
(b) `game_check_end`: si `wGameState ≠ GAME_JUGANDO`, ha acabado (`Z`). Si la oleada ha terminado (`wOleadaFin = 1`) y no quedan pollos vivos, `wGameState = GAME_GANADO` y ha acabado (`Z`). Si no, sigue (`NZ`).
(c) `game_pantalla_final`: `screen_off`, `sys_render_init`, la OAM a 0 (`memset`), un strongman (si has ganado) o un pollo (si has perdido) en `POS_FINAL` y `screen_on`. `main` hace `halt` después.

Pasos (intentadlo antes sin mirarlos)

1. (a) El recorrido de `game_entidad_en`, pero contando en vez de parar.
2. (b) Mirad primero `wGameState`, porque un pollo puede haber llegado a la casa en este mismo frame.
3. (c) `E` = tile elegido (`TILE_POLLO` o `TILE_STRONG`), `A = POS_FINAL`, `sys_render_pos_dir`, `ld a, e` y `sys_render_casilla`.

Consejos

- `Z` = ha acabado: `xor a`. `NZ` = sigue: `or 1`.
- (c) `sys_render_pos_dir` y `sys_render_casilla` no tocan `DE`.
- Un pollo muerto (`VIDA = 0`) sigue en el array hasta que `sys_render` lo libera: no lo contéis.

Si funciona

- Si un pollo llega a la casa, la pantalla queda con un pollo en el centro. Si derrotas a los 6 pollos, con un strongman.

Casos de prueba

Trazad vuestro código a mano con ellos. El resultado esperado está en el Anexo.

- **P1, has ganado.** `wGameState = 0`, `wOleadaFin = 1`. Única entidad: un pollo muerto (`VIDA = 0`).
- **P2, sigue.** `wGameState = 0`, `wOleadaFin = 0`. Un pollo vivo en `$54` y un strongman en `$52`. ¿Qué devuelve también `game_pollos_vivos`?

scn/scn_game_end.asm: game_pollos_vivos

```
;;-----------------------------------------------------------------
;; EJERCICIO 10 (a)
;;-----------------------------------------------------------------
;; Cuenta los pollos vivos: KIND_CHICKEN, CMP_ALIVE y VIDA != 0
;; Usa: a, bc, hl
;; Parametros entrada: -
;; Parametros salida: a-numero de pollos vivos
game_pollos_vivos::
    ;; ◉◉TU CÓDIGO◉◉
    xor a                       ;; (provisional: ninguno)
    ret
```

scn/scn_game_end.asm: game_check_end

```
;;-----------------------------------------------------------------
;; EJERCICIO 10 (b)
;;-----------------------------------------------------------------
;; Ha acabado la partida?
;;  - Si wGameState != GAME_JUGANDO: si (Z)
;;  - Si la oleada ha terminado (wOleadaFin = 1) y no quedan pollos vivos:
;;    wGameState = GAME_GANADO, si (Z)
;;  - Si no: no (NZ)
;; Usa: a, bc, hl
;; Parametros entrada: -
;; Parametros salida: Z, ha acabado. NZ, sigue
game_check_end::
```

```
    ;; ◉◉TU CÓDIGO◉◉
    or 1                        ;; (provisional: no acaba nunca)
    ret
```

scn/scn_game_end.asm: game_pantalla_final

```
;;-----------------------------------------------------------------
;; EJERCICIO 10 (c)
;;-----------------------------------------------------------------
;; Pantalla final: apaga el LCD, borra la pantalla (sys_render_init) y la OAM,
;; dibuja en POS_FINAL un strongman (GAME_GANADO) o un pollo (GAME_PERDIDO),
;; y enciende el LCD
;; Usa: a, bc, de, hl
;; Parametros entrada: -
;; Parametros salida: -
game_pantalla_final::
    ;; ◉◉TU CÓDIGO◉◉
    ret
```

Anexo · Resultados esperados

Miradlos solo después de trazar vuestro código
Son los valores comprobados con la solución de referencia en el emulador. «Entidad» = sus 6 bytes en orden `CMP, KIND, ESTADO, VIDA, DANO, POS`. Con `entity_array` en `$C000`, la entidad 1 empieza en `$C006`.

Caso	Resultado esperado
1 P1	`$70, $62` (bits bajos `01110000`, bits altos `01100010`)
1 P2	Cifra 0 en `$8100` (`$8000 + 16 × 16`). El último byte de la cifra 9 está en `$819F`.
2 P1	`E_ESTADO = $00`. Fondo en la casilla `$58`: `$9950, $9951 = 5, 5` y `$9970, $9971 = 6, 6`. Pollo en `$57`: `$994E, $994F = 7, 8` y `$996E, $996F = 9, 10`.
2 P2	La entidad queda libre (`CMP = 0`). Fondo en `$63`: `$9986, $9987 = 5, 5` y `$99A6, $99A7 = 6, 6`.
3 P1	Los 160 bytes de `$FE00-$FE9F` a `$00`.
3 P2	`$9800-$9809 = 11, 12, 11, 12, …` (5 veces) y `$9820-$9829 = 13, 14, ….` `$980A` y `$982A` sin cambiar (0).
4 P1	`112, 56, 15, $00 · 112, 64, 15, $20 · 120, 56, 15, $40 · 120, 64, 15, $60` (casilla fila 6, columna 3).
4 P2	`16, 72, 15, $00 · 16, 80, 15, $20 · 24, 72, 15, $40 · 24, 80, 15, $60` (casilla fila 0, columna 4).
5 P1	`Z`, `HL = entity_array + 6` (`$C006`), `B = $57`.
5 P2	`NZ` (el strongman está muerto), `B = $52`.
6 P1	`NC`, `wProtein = 7` (coste del tipo 1: 3). Entidad 0: `$81, $02, $04, $14, $02, $52` (tipo 1: estado 4, vida 20, daño 2, casilla `$52`).
6 P2	`C`, `wProtein = 2` y ninguna entidad creada.
6 P3	`$9A20, $9A21 = 16, 19` («03») y `$9A32, $9A33 = 16, 23` («07»).
7 P1	`wOleadaEspera = 180`, `wOleadaFin = 0`, `wOleadaPtr = Oleada1 + 1`.
7 P2	Entidad 0: `$83, $01, $00, $05, $02, $79` (pollo B en el carril 2, columna 9). `wOleadaEspera = 120`, `wOleadaPtr` avanza 3 y `wOleadaFin = 0`.
7 P3	`wOleadaEspera = 4` y nada más.
8 P1	Pollo: `E_POS = $53`, `E_ESTADO = $80`. `wGameState = 0`.
8 P2	`wGameState = 1` (`GAME_PERDIDO`). El pollo sigue en `$50`.
8 P3	Nada cambia: el pollo sigue en `$54` con `E_ESTADO = 0`.
9 P1	Pollo `VIDA = 8`. Strongman `VIDA = 19`. `wProtein = 10`.
9 P2	Pollo muerto: `CMP = $81` y `VIDA = 0`. Strongman `VIDA = 19`. `wProtein = 12`.
10 P1	`Z`, `wGameState = 2` (`GAME_GANADO`).
10 P2	`NZ`, `wGameState = 0`. `game_pollos_vivos` devuelve `A = 1`.

Cuando esté todo
Con los 10 ejercicios, el juego se puede jugar entero: proteína inicial 10, 6 pollos y strongmen de 5 tipos (coste 2-6). Ideas para seguir, ya fuera del challenge: más oleadas, un menú de inicio (gestor de escenas del Ch. 5), animación con `E_ESTADO`, ataques a distancia o sonido.